E10 — Course Schedule (Topological Sort / Cycle Detection)

Experiment: 10 | **LeetCode:** #207 | **CO:** CO3, CO4 | **BT Level:** BT5

Problem Statement

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

```
Input:  numCourses = 2, prerequisites = [[1,0]]
Output: true
Explanation: Take course 0 first, then course 1.
```

Example 2:

```
Input:  numCourses = 2, prerequisites = [[1,0],[0,1]]
Output: false
Explanation: Cycle: course 0 requires course 1 and vice versa.
```

Concept: Course Schedule as a Directed Graph

Model the problem as a **Directed Graph:**

- Node = course
- Edge `b → a` = course `b` must be taken before course `a`

Question: Can we finish all courses?
Equivalently: Does the directed graph have a cycle?
Answer: If the graph is a **DAG (Directed Acyclic Graph)** → `true`; else → `false`.

Solution 1: DFS Cycle Detection (3-Colour)

Use 3 states for each node:

- `0` = Unvisited (white)
- `1` = In current DFS path (grey) — currently being processed
- `2` = Fully processed (black)

A **cycle exists** if we encounter a grey node during DFS (back edge).

```
from collections import defaultdict
```

```
def canFinish_dfs(numCourses, prerequisites):
    """
    DFS-based cycle detection.
    Time: O(V + E)  |  Space: O(V + E)
    """
    graph = defaultdict(list)
    for course, prereq in prerequisites:
        graph[prereq].append(course)   # prereq → course

    state = [0] * numCourses   # 0=white, 1=grey, 2=black

    def has_cycle(node):
        if state[node] == 1:   # Back edge → cycle!
            return True
        if state[node] == 2:   # Already fully processed
            return False

        state[node] = 1   # Mark as in-progress (grey)

        for neighbor in graph[node]:
            if has_cycle(neighbor):
                return True

        state[node] = 2   # Mark as done (black)
        return False

    for course in range(numCourses):
        if state[course] == 0:
            if has_cycle(course):
                return False

    return True
```

Solution 2: Kahn's Algorithm (BFS Topological Sort)

In-degree approach:

1. Compute **in-degree** (number of prerequisites) for each course.
2. Add all courses with in-degree 0 to the queue.
3. Process queue: for each course popped, reduce in-degree of its dependents.
4. If a dependent reaches in-degree 0, add to queue.
5. If all `numCourses` are processed → no cycle → `true`.

```
from collections import deque, defaultdict

def canFinish_kahn(numCourses, prerequisites):
    """
    Kahn's BFS-based topological sort.
    Time: O(V + E)  |  Space: O(V + E)
    """
    graph = defaultdict(list)
```

```
    in_degree = [0] * numCourses

    for course, prereq in prerequisites:
        graph[prereq].append(course)
        in_degree[course] += 1

    # Start with all courses that have no prerequisites
    queue = deque([c for c in range(numCourses) if in_degree[c] == 0])
    processed = 0

    while queue:
        course = queue.popleft()
        processed += 1

        for dependent in graph[course]:
            in_degree[dependent] -= 1
            if in_degree[dependent] == 0:
                queue.append(dependent)

    return processed == numCourses
```

Detailed Trace

Example 1: `numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]`

Graph (prereq → course):

```
0 → [1, 2]
1 → [3]
2 → [3]
```

In-degrees: `[0, 1, 1, 2]`

Kahn's BFS:

```
Queue: [0]   (in_degree[0] = 0)
processed = 0

Pop 0: processed=1
  Neighbor 1: in_degree[1] = 1-1 = 0 → enqueue
  Neighbor 2: in_degree[2] = 1-1 = 0 → enqueue
Queue: [1, 2]

Pop 1: processed=2
  Neighbor 3: in_degree[3] = 2-1 = 1 (not 0)
Queue: [2]

Pop 2: processed=3
  Neighbor 3: in_degree[3] = 1-1 = 0 → enqueue
Queue: [3]
```

```
Pop 3: processed=4
  No neighbors
Queue: []

processed=4 == numCourses=4 → return True ✓
```

Example 2: `numCourses = 2, prerequisites = [[1,0],[0,1]]`

Graph: `0 → [1], 1 → [0]`
In-degrees: `[1, 1]`

Kahn's BFS:

```
Queue: []   (no course with in_degree = 0)

Queue is empty immediately.
processed = 0 ≠ numCourses = 2 → return False ✓
```

DFS Trace for Example 2:

```
state = [0, 0]
graph: 0 → [1], 1 → [0]

has_cycle(0):
  state[0] = 1 (grey)
  Neighbor 1:
    has_cycle(1):
      state[1] = 1 (grey)
      Neighbor 0:
        state[0] == 1 → CYCLE DETECTED → return True
    returns True
  returns True

canFinish returns False ✓
```

Topological Order Variant

```
def findOrder(numCourses, prerequisites):
    """
    Return actual topological order (LeetCode #210).
    Returns [] if cycle exists.
    """
    graph = defaultdict(list)
    in_degree = [0] * numCourses

    for course, prereq in prerequisites:
        graph[prereq].append(course)
        in_degree[course] += 1

    queue = deque([c for c in range(numCourses) if in_degree[c] == 0])
```

```
    order = []

    while queue:
        course = queue.popleft()
        order.append(course)
        for dep in graph[course]:
            in_degree[dep] -= 1
            if in_degree[dep] == 0:
                queue.append(dep)

    return order if len(order) == numCourses else []
```

Test Suite

```
def test_can_finish():
    test_cases = [
        (2, [[1,0]], True),
        (2, [[1,0],[0,1]], False),
        (1, [], True),
        (4, [[1,0],[2,0],[3,1],[3,2]], True),
        (3, [[0,1],[0,2],[1,2]], True),
        (3, [[1,0],[2,1],[0,2]], False),   # Cycle: 0→1→2→0
    ]

    for n, prereqs, expected in test_cases:
        r1 = canFinish_dfs(n, prereqs)
        r2 = canFinish_kahn(n, prereqs)
        for r, name in [(r1, "dfs"), (r2, "kahn")]:
            status = "PASS" if r == expected else "FAIL"
            print(f"{status} [{name}]: canFinish(n={n}, prereqs={prereq

test_can_finish()
```

Complexity Analysis

Solution	Time	Space	Notes
DFS (3-color)	O(V + E)	O(V + E)	Recursive; handles large graphs with sys.setrecursionlimit
Kahn's BFS	O(V + E)	O(V + E)	Iterative; produces topological order

---

Key Takeaways

- **Course scheduling = cycle detection in a DAG.**
- **DFS:** A cycle exists if we revisit a grey (in-progress) node — back edge in DFS tree.
- **Kahn's BFS:** If topological sort processes fewer than `V` nodes, a cycle exists.
- Both solutions are O(V + E) — optimal for this problem.
- The 3-color DFS is a fundamental technique; Kahn's BFS also produces the topological

ordering.

References

- LeetCode #207 — Course Schedule
- LeetCode #210 — Course Schedule II (topological order)
- L32.md (DFS) — DFS and topological sort theory
- Cormen et al., *Introduction to Algorithms*, Ch. 22.3 (Topological Sort), Ch. 22.4 (Strongly Connected Components)